

Xinu **xinu-rpi4** User's Manual

Embedded Xinu for the Raspberry Pi 4 (BCM2711 / Cortex-A72, AArch64)

Embedded Xinu port for the Raspberry Pi 4 (BCM2711 / Cortex-A72, AArch64). The same source tree also cross-builds for the QEMU `virt` machine. This manual is written for **operators** — for the development roadmap see `README.md`.

Contents

1	Targets and Build Artefacts	1
1.1	Feature Matrix (read this first!)	1
1.2	Boot Sequence (the unusual bits)	2
2	What You Need	2
2.1	Hardware (for a real board)	2
2.2	Host software	2
3	Build	2
3.1	Basic commands	3
3.2	Toolchain auto-detection	3
3.3	Key Make variables	3
3.4	Per-target macros (passed via <code>CFLAGS</code>)	3
3.5	Linker scripts	4
4	Writing the SD Card	4
4.1	Pi 4	4
4.2	Manual copy (e.g. external SD reader)	4
4.3	What else must be on the card	4
5	Serial Console	4
6	Shell Command Reference	5
6.1	procdemo — real context switch	5
6.2	pingpong vs procdemo	5
7	Networking (Pi 4)	6
7.1	Ping from a Mac	6
7.2	DHCP / TCP status	6
7.3	WiFi (BCM43455)	6
7.4	Mesh networking: multiple Xinu nodes (MANET ad-hoc)	7

8	Running under QEMU	7
9	Important config.txt Lines	7
10	USB / Input Device Support	8
11	Troubleshooting	8
12	Directory Layout (operator view)	9
13	Recipe Book	9
14	References	9
15	License	9

1 Targets and Build Artefacts

Target	Board / machine	Output image	UART0 base
pi4	Raspberry Pi 4 (BCM2711)	kernel8.img	0xFE201000
qemu	QEMU -M virt -cpu cortex-a76	kernel_virt.img	0x09000000

The load address is **0x80000** for Pi 4 and **0x40080000** for QEMU. Both images come from one source tree, distinguished by `-D<TARGET>` macros and a per-target linker script.

1.1 Feature Matrix (read this first!)

Not every feature works on every target. Several subsystems run only on real hardware (Pi 4):

Subsystem	Pi 4	QEMU	Notes
PL011 UART0 shell	✓	✓	The common substrate
Cooperative scheduler (S0)	✓	✓	procdemo / pingpong
Preemptive scheduler (S1)	✓	WIP	100 Hz timer tick works on Pi 4 (ticks)
HDMI framebuffer	✓	—	Usable on Pi 4
Window manager	✓	—	1280×960 virtual desktop, Pi 4 only
SD card + FAT32	✓	—	EMMC driver is BCM2711-specific
Ethernet (GENET)	✓	—	NET-E done (BCM2711 GENET)
USB (keyboard / HID)	✓	—	Via VL805 PCIe xHCI (USB-2.0 ports)
DHCP / TCP	✗	—	Code present but dispatch OFF (§7.4)

! The source tree contains `device/genet/`, `device/sd/`, `device/usb/xhci/` etc., initialised against **Pi 4 (BCM2711) register addresses**. The `-D<BASE>` macros (`GENET_BASE`, ...) are defined only for the Pi 4 build; the QEMU build leaves them undefined so the corresponding code is linked out or no-ops.

1.2 Boot Sequence (the unusual bits)

`loader/boot.S` does more than a textbook bare-metal stub:

1. **Mandatory Linux ARM64 Image header.** The Pi 4 EEPROM bootloader can refuse to jump into the kernel unless it sees the magic "ARM\X64" at file offset 0x38. `boot.S` places a 64-byte header right at `_start`:. Symptom of a missing header: the firmware rainbow pattern stays on HDMI and your code never runs.
2. **Pin secondary cores with MPIDR_EL1.** The boot core continues; cores 1/2/3 enter a `wfe` park loop until S0/S1 wakes them.
3. **Set up SP using the leex convention.** `sp = _start (= 0x800000)` — the kernel base address doubles as the initial stack pointer.
4. **Preserve the DTB pointer.** Firmware passes the device tree physical address in `x0`; the stub stashes it across BSS clear and writes it back to `.data dtb_addr`.
5. **Zero BSS, then jump to C** (`bl kernel_main`). If `kernel_main` ever returns, drop into the same `wfe` park loop.

The stub does **not** implement an EL2 → EL1 transition (same assumption holds for QEMU). Adding `armstub=` or `kernel_old=1` would change this.

2 What You Need

2.1 Hardware (for a real board)

- Raspberry Pi 4 (BCM2711)
- microSD card with a FAT32 `bootfs` partition (easiest: flash Raspberry Pi OS first, then overwrite `kernel8.img` and `config.txt`)
- 3.3 V USB-serial adapter, 115200 8N1 — header pin 8 (TXD → GPIO14), pin 10 (RXD → GPIO15), and a GND (e.g. pin 6)
- For networking: an Ethernet cable

2.2 Host software

macOS or Linux, plus an AArch64 cross-toolchain (pick one):

```
brew install aarch64-elf-gcc      # GNU
brew install --cask gcc-arm-embedded # ARM official
```

`qemu-system-aarch64` for the QEMU target (`brew install qemu`), and a serial terminal client (`screen`, `minicom`, `picocom`, ...).

3 Build

Always run `make` from `compile/` — the Makefile uses `VPATH` to gather sources from `../loader/`, `../device/...`, etc.

3.1 Basic commands

```
cd /Users/kodamay/projects/xinu-rpi4/compile

make pi4          # -> compile/kernel8.img      (real Pi 4)
make qemu         # -> compile/kernel_virt.img  (QEMU virt)
```

```
make                # = make all = pi4 + qemu
make clean          # remove all objects and images
```

Objects live in per-variant trees (`obj/pi4/`, `obj/qemu/`), so building different targets in sequence only recompiles what differs.

3.2 Toolchain auto-detection

The Makefile looks for AArch64 GCC as `$(GCCPATH)/bin/aarch64-elf-gcc` then `aarch64-none-elf-gcc`. Default `GCCPATH` is `/opt/homebrew`:

```
make pi4 GCCPATH=$HOME/aarch64/arm-gnu-toolchain-...-aarch64-none-elf
make pi4 CROSS=aarch64-linux-gnu-
```

3.3 Key Make variables

Variable	Default	Purpose
<code>GCCPATH</code>	<code>/opt/homebrew</code>	Cross-toolchain install prefix
<code>CROSS</code>	(auto-detected)	Tool prefix (<code>aarch64-elf-</code> , ...)
<code>SDCARD</code>	<code>/Volumes</code>	Parent mount point for <code>make install_*</code>
<code>DEST</code>	<code>\$(SDCARD)/bootfs</code>	Target FAT32 partition

3.4 Per-target macros (passed via CFLAGS)

Macro	Pi 4	QEMU virt
<code>-mcpu</code>	<code>cortex-a72</code>	<code>cortex-a76</code>
<code>UART0_BASE</code>	<code>0xFE201000</code>	<code>0x09000000</code>
<code>MBOX_BASE</code>	<code>0xFE00B880</code>	(undef)
<code>SD_BASE</code>	<code>0xFE340000</code>	—
<code>USB_BASE</code>	<code>0xFE980000</code>	—
<code>GIC_BASE</code>	<code>0xFF840000</code>	—
<code>PCIE_BASE</code>	<code>0xFD500000</code>	—
<code>GENET_BASE</code>	<code>0xFD580000</code>	—
<code>HEAP_END</code>	<code>0x40000000</code> (1G)	<code>0x50000000</code>
<code>SKIP_MBOX</code>	—	defined (simplify)
<code>KERNEL_NAME</code>	<code>"kernel8.img"</code>	<code>"kernel_virt.img"</code>
<code>BOARD_NAME</code>	<code>"Pi4"</code>	<code>"virt"</code>
<code>SOC_NAME</code>	<code>"BCM2711"</code>	<code>"QEMU"</code>

Common CFLAGS: `-Wall -O2 -ffreestanding -nostdinc -nostdlib -nostartfiles -mgeneral-regs-only`. The flag keeps most code FPU/NEON-free; only `cc.c` and `llm.c` are built with FP enabled (CPACR releases the FPU at EL1).

3.5 Linker scripts

Script	Target	Entry / load address
<code>compile/link.ld</code>	Pi 4	<code>0x80000</code>
<code>compile/link_virt.ld</code>	QEMU virt	<code>0x40080000</code>

Both place `.text.boot` at the start and export `__bss_start / __bss_size` (8-byte units), following the leex convention.

4 Writing the SD Card

4.1 Pi 4

```
cd compile
make install_pi4 SDCARD=/Volumes
# -> copies kernel8.img and sdcard/config_pi4.txt to /Volumes/bootfs
```

`make install` is an alias for `install_pi4`.

4.2 Manual copy (e.g. external SD reader)

```
diskutil mount /dev/disk4s1          # -> /Volumes/bootfs
cp compile/kernel8.img /Volumes/bootfs/
cp sdcard/config_pi4.txt /Volumes/bootfs/config.txt
sync
diskutil eject /Volumes/bootfs
```

! Verify the md5 of `kernel8.img` on the card matches `compile/` — booting an old image is the most common time sink.

4.3 What else must be on the card

`kernel8.img` + `config.txt` alone won't boot. The same FAT32 partition also needs the Raspberry Pi OS firmware blobs (`bootcode.bin`, `start4.elf`, `fixup4.dat`, ...). Fastest path: flash Pi OS first, then overwrite only `kernel8.img` and `config.txt`.

5 Serial Console

```
screen /dev/tty.usbserial-XXXX 115200          # macOS
minicom -b 115200 -o -D /dev/tty.usbserial-XXXX # macOS
sudo screen /dev/ttyUSB0 115200                 # Linux
```

Exit `screen` with `Ctrl-a k` then `y`. Within 5 seconds of power-on you should see:

```
=====
Xinu Pi4 hello (AArch64, BCM2711, kernel8.img)
PL011 UART0 @ 0xFE201000, 115200 8N1
bootstrap: leex-style stub + xinu-rpi4 main
=====

Round 1 phase B/U done -- entering interactive shell.
type `help` for the command list.
xinu-pi4$ _
```

6 Shell Command Reference

Command	Purpose
help / ?	List registered commands
echo <words>	Echo args back (whitespace-collapsed)
hello	Smoke marker — greeting
mem	Show <code>__bss_start</code> / <code>__bss_end</code> / <code>_end</code>
peek <hex_addr>	Read a 32-bit MMIO word (e.g. <code>peek 0xfe201018</code>)
uptime	Raw <code>CNTPCT_EL0</code> (generic timer counter)
ticks	100 Hz timer tick counter (S1)
ps	Core / EL status
halt	Mask DAIF + PSCI <code>SYSTEM_OFF</code> (QEMU exits cleanly)
reboot	Watchdog-driven reset via the BCM2711 PM block
pingpong [N]	AIPL-style 2-actor cooperative PingPong, N=1..50
procdemo [N]	Real 2-process ctxsw demo, N=1..30
usb	DWC2 USB HCD diagnostics (Pi 4 only)
rxstat	Drain RX ring + show packet / byte counters
pan <dx> <dy>	Scroll the virtual 1280×960 desktop viewport
view	Show viewport / desktop sizes
autopan [on off]	Toggle demo auto-scroll

6.1 procdemo — real context switch

```
xinu-pi4$ procdemo 3
procdemo: created pid=1 (ping) and pid=2 (pong), iters=3
[Ping pid=1] tick 1
[Pong pid=2] tock 1
[Ping pid=1] tick 2
[Pong pid=2] tock 2
[Ping pid=1] tick 3
[Pong pid=2] tock 3
[Ping pid=1] exit at iter 3
[Pong pid=2] exit at iter 3
procdemo: both processes exited; back in shell.
```

Each `pid=N` is read live from global `currpids`, so the alternation is direct evidence the scheduler flipped contexts. Stacks are not reclaimed on `proc_exit` yet — S1 (clock IRQ) will let the dispatcher reap them.

6.2 pingpong vs procdemo

Aspect	pingpong	procdemo
Actors	Two static (Ping, Pong)	Real procs via <code>proc_create</code>
Switch	Single-stack dispatcher loop	Real ctxsw via <code>ctxsw.S</code>
Terminate	Both inboxes empty	Both call <code>proc_exit()</code>

7 Networking (Pi 4)

The network stack runs on the Pi 4 GENET (BCM2711).

What works: ARP request/reply; ICMP echo (ping); static IP/MAC **192.168.3.100** / **d8:3a:dd:a7:fd:bf** (set in **loader/main.c:707** area); raw broadcast/unicast TX; 16-slot RX ring.

7.1 Ping from a Mac

```
sudo arp -s 192.168.3.100 d8:3a:dd:a7:fd:bf
ping 192.168.3.100
```

RTT is 900 ms (rate-limited by the window-manager tick). Quick shell checks: **rxstat** and **ticks**.

7.2 DHCP / TCP status

system/dhcp_client.c and **system/tcp_server.c** ship in the tree but their **rx_tick** dispatch is **OFF**: repeated DHCP broadcast TX degrades the GENET TX ring and stalls ICMP, and routing **tcp_handle_packet()** from **rx_tick** stops subsequent ICMP echo replies. Re-enabling is planned in a future sprint (NET-G bisect).

7.3 WiFi (BCM43455)

The Pi 4 has an on-board BCM43455 WiFi chip (separate from the wired GENET above). It is driven entirely from the serial shell (**xinu-pi4\$**): bring up the firmware, scan, then connect. It does **not** auto-connect at boot.

Command	What it does
wifi probe	M0/M1 bring-up: download the firmware to the chip (once per boot)
wifi scan	escan for nearby APs
wifi up <ssid> <pass>	join (WPA2-PSK) + DHCP; starts the persistent ARP/ICMP responder
wifi join <ssid> <pass>	join only (no DHCP)
wifi dhcp	request a DHCP lease
wifi off	radio down / disconnect
wifi ping <a.b.c.d> [count]	ICMP echo client
wifi serve [secs]	run the ARP/ICMP responder for N seconds (default 30)
wifi resolve <host> / wifi web <host>	DNS / DNS + HTTP GET
wifi ntp [a.b.c.d]	NTP time client
wifi tftp <ip> <file> / wifi netboot <ip> <file>	TFTP fetch / fetch + chainload

```
xinu-pi4$ wifi probe          # download firmware (once per boot)
xinu-pi4$ wifi scan           # see what's around
xinu-pi4$ wifi up MyHome-5G mypassword
wifi up: connected; ARP/ICMP responder is now persistent.
xinu-pi4$ wifi ping 8.8.8.8
```

WiFi does not auto-reconnect after a reboot — re-run **wifi probe** + **wifi up** each boot. **wifi netboot <ip> <file>** fetches a kernel over WiFi and chainloads it (a no-SD-swap update path, like the wired chainload).

7.4 Mesh networking: multiple Xinu nodes (MANET ad-hoc)

Several Pi 4 (and Pi 3) Xinu boards can form a peer-to-peer mesh with **no access point**, using 802.11 IBSS (ad-hoc) mode plus on-demand AODV routing — the same MANET stack used in the drone-HIL demo (Pi 4 + Pi 3 nodes). Each node joins the same ad-hoc cell with a distinct node number:

```
xinu-pi4$ wifi probe                # once per boot
xinu-pi4$ wifi adhoc <cell-ssid> [ch] [node]
```

- **<cell-ssid>** — the cell name; **all nodes must use the same name and channel**.
- **[ch]** — 802.11 channel (default 6).
- **[node]** — this node's number (default 1); it gets the static IP **10.0.0.<node>/24**. **Give every node a distinct number**.

Example — a 3-node cell on channel 6:

```
xinu-pi4$ wifi adhoc mesh1 6 1      # -> 10.0.0.1
xinu-pi4$ wifi adhoc mesh1 6 2      # -> 10.0.0.2
xinu-pi4$ wifi adhoc mesh1 6 3      # -> 10.0.0.3
```

Nodes within radio range reach each other directly at **10.0.0.x** (**wifi ping 10.0.0.2**).

Multi-hop (AODV). When a destination is not in direct range, discover a route on demand with **wifi aodv <ip>**:

```
xinu-pi4$ wifi aodv 10.0.0.3
[wifi] === AODV discover 10.0.0.3 ===
[wifi] aodv: RREQ id=1 for 10.0.0.3
[wifi] *** AODV route: 10.0.0.3 via 10.0.0.2, 2 hop ***
```

The minimal AODV module (M13, RFC 3561 core) broadcasts an RREQ (UDP port 654); each in-range node relays it and records a reverse route; the destination answers with an RREP that installs the forward route. Every node relays for its peers automatically (route table: up to 16 entries). IBSS / ad-hoc is independent of the infrastructure **wifi up** mode (run **wifi off** to leave an AP first) and is not restored after reboot — re-run **wifi probe + wifi adhoc** on each node.

8 Running under QEMU

```
cd compile
make qemu          # interactive -- Ctrl-A X to quit
make qemu-smoke    # canned input, writes qemu-smoke.log
```

Under QEMU: **MIDR_EL1 = 0x414fd0b1** (QEMU's published Cortex-A76 part number; real Pi 4 hardware runs Cortex-A72); **halt** cleanly exits because **virt** handles PSCI **SYSTEM_OFF**; no networking, USB, SD, or HDMI.

9 Important config.txt Lines

sdcard/config_pi4.txt (copied to the card's **config.txt** by **make install_pi4**):

Line	Meaning
arm_64bit=1	Boot in AArch64
kernel=kernel8.img	Pi 4 kernel name
kernel_address=0x80000	Load address
enable_uart=1	Enable UART
uart_2ndstage=1	Print second-stage bootloader log on UART
dtparam=uart0=on	Route UART0 to GPIO14/15
init_uart_clock=48000000	Pin the PL011 reference clock to 48 MHz
hdmi_force_hotplug=1	Force HDMI hotplug
framebuffer_width=640	Width of firmware-allocated framebuffer
framebuffer_height=480	Height
framebuffer_depth=32	bpp

The KMS overlay (`dtoverlay=vc4-kms-v3d`) is left in — Xinu still consumes the firmware’s simple-framebuffer via the VC mailbox.

10 USB / Input Device Support

The Pi 4’s USB-A ports go through **VL805 (PCIe xHCI)**; a hand-rolled xHCI driver brings up USB keyboards and mice.

- **Use the USB-2.0 (black) ports.** They route through the USB-2.0 hub (Genesys, working TT), so low-speed interrupt-IN split transactions are delivered. The USB-3.0 (blue) ports sit behind the VIA USB3 hub whose path does not deliver the periodic transfer (the cursor won’t move).
- The shell’s `usb` command is a diagnostic dump for the DWC2 (USB-C OTG) HCD, not a general device-control interface.
- VL805 firmware leaves the PCIe RC in reset on an SD boot, so boot issues a `NOTIFY_XHCI_RESET` mailbox tag (`0x00030058`, devid `0x100000`) to bring xHCI up (details in `NEXT_SESSION_PI4.md`).

Board	Input	Output
Pi 4	USB-A keyboard / UART0 serial	UART0 serial + HDMI framebuffer
QEMU	qemu stdio (serial)	qemu stdio (serial)

11 Troubleshooting

Symptom	What to check
No output on serial	device name + speed in <code>screen</code> ; TX/RX swapped; GND
Banner appears, then stops	Old <code>kernel8.img</code> on SD — rebuild and recopy
No echo at the prompt	Local echo on the host terminal — keep it OFF
<code>peek</code> returns <code>0xFFFFFFFF</code>	Address not mapped; MMU is flat ID
USB keyboard/mouse dead	Make sure it’s on a USB-2.0 (black) port (§11)
Ping fails on Pi 4	<code>sudo arp -s ... done?</code> ; cable; <code>rxstat</code> increments?
<code>make qemu</code> fails	Check <code>qemu-system-aarch64</code> version (≥ 11)
<code>make install</code> permission	It uses <code>sudo cp</code> — provide the sudo password
Reboot loop on power-on	Does <code>kernel=</code> match the file on the card?
Rainbow pattern stays on HDMI	Likely missing ARM64 Image header — rebuild

Generic recovery flow:

```
cd compile && make clean && make pi4
make install_pi4 SDCARD=/Volumes
diskutil eject /Volumes/bootfs
# Reseat the SD card and power-cycle the board.
```

12 Directory Layout (operator view)

```
xinu-rpi4/
├── compile/                # run `make` here
│   ├── Makefile
│   ├── kernel8.img        # Pi 4 (built by `make pi4`)
│   └── kernel_virt.img    # QEMU (built by `make qemu`)
├── sdcard/
│   └── config_pi4.txt      # Pi 4 (copied by install_pi4)
├── README.md              # Developer-facing roadmap + internals
├── USERS_MANUAL_EN.md     # Markdown source of this manual
├── USERS_MANUAL_JA.md     # Japanese version
└── NEXT_SESSION_PI4.md    # Handoff notes for ongoing work
```

13 Recipe Book

From a fresh checkout to a running Pi 4:

```
cd /Users/kodamay/projects/xinu-rpi4/compile
make clean && make pi4
make install_pi4 SDCARD=/Volumes
diskutil eject /Volumes/bootfs
screen /dev/tty.usbserial-XXXX 115200
```

Quick command sanity check (QEMU):

```
cd compile && make qemu
xinu-pi4$ procdemo 5
xinu-pi4$ pingpong 3
xinu-pi4$ halt          # Ctrl-A X to exit QEMU
```

14 References

- Upstream Xinu (32-bit): <https://github.com/yaskodama/xinu-rpi>
- AArch64 boot stub origin: <https://github.com/radlyeel/leex>
- Shell centry pattern origin: <https://github.com/davidxyz/xinuPi>
- Development roadmap (Round 1 plan): under `aice-pi-evolution/experiments/` in the `abclcp-project` repo

15 License

Inherits the BSD-style licenses of upstream Xinu and leex. See **LICENSE** once the source-of-truth file is added.